

Multer Storage Concepts

in backend to take form-data in request it will not possible with `express.json()` we need to use multer package but in that can you explain me concept of disk storage or memory storage and why we need storage in that ?

Great question 👍 You're thinking in the right direction.

Yes — `express.json()` and `express.urlencoded()` cannot handle `multipart/form-data` (used for file uploads). That's why we use **Multer**.

Now let's understand the **core concept of storage in Multer** 📌

♦ Why Do We Need Storage in Multer?

When a client sends `multipart/form-data`, it can contain:

- Text fields
- Files (images, PDFs, videos, etc.)

Multer's job is:

1. Parse the incoming request
2. Extract files
3. Decide **where and how to store those files**

👉 Files must go somewhere temporarily or permanently.

That "somewhere" is defined by **storage engine**.

Without storage:

- Multer wouldn't know where to keep the uploaded file.
- The file would just stay in memory stream and disappear.

So storage tells Multer:

- Should I save file on disk?
- Should I keep it in RAM?
- What should be the filename?
- Where should I store it?

◆ Two Types of Storage in Multer

1 Disk Storage (Permanent / Physical Storage)

JavaScript

```
const multer = require("multer")

const storage = multer.
    function req file cb
    cb null "uploads/"

    function req file cb
    cb null Date.    + "-" + file.

const upload = multer    storage
```

🔍 What Happens Here?

- `destination` → folder where file will be saved
- `filename` → how file will be named
- File is saved physically on your server

📌 Use Case:

- Image upload
- PDF upload
- Profile picture
- Any file you want to store permanently

📁 Result:

File will appear inside:

Code

2 Memory Storage (RAM Storage)

JavaScript

```
const storage = multer.  
const upload = multer      storage
```

What Happens Here?

- File is stored in **RAM (Buffer)**
- Not saved to disk
- Available in:

JavaScript

```
req. .
```

Use Case:

- When uploading file directly to:
 - AWS S3
 - Cloudinary
 - Firebase Storage
- When you don't want to save file locally

Important:

If server crashes → file gone
Because it was in memory only.

Disk vs Memory Comparison

Feature	Disk Storage	Memory Storage
Stored where?	Hard disk	RAM
Permanent?	Yes	No
Speed	Slower (write to disk)	Faster
Good for	Local file saving	Cloud upload
Risk	Uses disk space	Uses server memory

When Should YOU Use What?

✓ Use Disk Storage when:

- You want file inside your project
- You are building:
 - College project
 - Local server
 - Basic backend

✓ Use Memory Storage when:

- Upload → directly send to cloud
- You don't want temporary file on server
- Microservice architecture

Why Not Just Use `express.json()`?

Because:

```
http
```

is completely different from:

```
http
```

```
express.json()
```

 only parses JSON body

It cannot handle file streams.

Multer handles:

- Streams
- Boundaries
- Binary data

Internally What Happens?

When file is uploaded:

1. Request comes
2. Multer reads stream chunk by chunk
3. Converts file into:
 - Disk file (if diskStorage)
 - Buffer (if memoryStorage)
4. Adds file info into:

JavaScript

```
req.  
req.
```

Final Concept in One Line

 Storage in Multer defines where and how uploaded files should be stored (disk or memory).

If you want, I can also explain:

- How Multer handles multiple files
- How to add file type validation
- How to limit file size
- Or how Multer works internally with streams

Just tell me 😊

you mean when multer after converting file from form-data and add in to req.file that express can read it ?
and one more question is this server or my local machine is this same think like you tell me File is saved physically on your server explain it properly server in mean in database or something else ?

Very good questions 🔥 You're thinking like a backend engineer now.

Let's clear both doubts properly.

✅ 1 After Multer processes file, can Express read it?

Yes ✓

But small correction:

👉 Multer **does not convert file into JSON**.

It **parses multipart/form-data** and extracts:

- Text fields → `req.body`
- Files → `req.file` or `req.files`

Example:

JavaScript

```
app.    "/upload" upload.    "photo" req res
  console. req.    // file info
  console. req.    // text fields
```

So what happens internally?

1. Client sends `multipart/form-data`
2. Multer intercepts request BEFORE your route handler
3. It:
 - Reads file stream
 - Stores it (disk or memory)
 - Attaches metadata to `req.file`
4. Then Express route runs
5. Now you can access it

So yes — after Multer adds file to `req.file`, Express can read it because Multer modifies the request object before your route handler executes.

✅ 2 What do you mean by "server"? Is it my local machine?

Very important concept 👉

- ♦ What is Server?

A **server** is just a machine running your backend code.

It can be:

1. Your Local Machine

When you run:

```
Bash
```

```
node
```

Your laptop becomes the server.

If using diskStorage:

```
JavaScript
```

```
destination "uploads/"
```

File will be saved in:

```
Code
```

👉 That means it is saved in YOUR computer.

2. Remote Server (Production)

When deployed to:

- AWS EC2
- DigitalOcean
- VPS
- Render
- Railway

Then file is saved in THAT machine's disk.

Not your laptop.

 **Very Important: Server ≠ Database**

When we say:

"File is saved on server"

We mean:

👉 It is saved in the **file system (hard disk)**

NOT inside database.

◆ File Storage vs Database

📁 Disk Storage

Saved like:

Code

It is a real file in folder.

🗄 Database Storage

You have 2 ways:

1. Store file path in DB:

JavaScript

```
username "Meet"  
profilePic "uploads/170870000-image.png"
```

2. Store file as binary (Buffer / BLOB)
(not recommended for large files)
-

🧠 So Where Exactly Is File Saved?

If using diskStorage:

JavaScript


```
destination function req file cb
cb null "uploads/"
```

File is saved inside:

Code

This folder is inside:

- Your local system (if running locally)
- OR
- Your hosting server machine (if deployed)

Real World Example

Think like this:

- Your laptop = small office server
- AWS machine = big rented office
- Database = locker
- Disk storage = cupboard
- Memory storage = table (temporary)

If you put file on table (memoryStorage)

→ It disappears after restart

If you put file in cupboard (diskStorage)

→ It stays there

If you put file in locker (database BLOB)

→ It stays in database

Final Clean Summary

✓ Multer adds file info to `req.file`

Then Express can access it.

✓ "Server" means machine running your backend

- Local machine (during development)
- Cloud machine (after deployment)

✓ Disk storage means:

File is saved in file system (NOT database)

If you want next level explanation I can also explain:

- What happens if server restarts?
- Why we usually upload directly to S3 instead of server?
- What is ephemeral storage in cloud?

Just tell me 😊

okay my final goal is user will upload the file and i have to access it and now let suppose i host the backend on render now if i store that file in disk it will affect to costing to save from i use cloud storage like imagekit and aws but my cost will increase strong file on render server just explain me how charges apply on bandwidth how bandwidth become larger ?

Perfect 🔥 now you're thinking about **real production architecture + cost optimization**.

This is very important when deploying on **Render** or any cloud platform.

Let's break this properly and clearly.

🎯 Your Situation

- User uploads file
- Backend hosted on **Render**
- You are deciding:
 - Store file on Render disk?
 - Or use cloud storage like **Amazon Web Services (S3)** or **ImageKit**?

And you're worried about:

- 💰 Cost
- 🌐 Bandwidth
- 📦 Storage

Good. Let's understand everything step-by-step.

♦ 1 If You Store Files on Render (Disk Storage)

When using:

```
JavaScript
```

```
multer.
```

File gets saved on Render's server disk.

🚨 Important Problem on Render

Render free and many paid plans use:

👉 Ephemeral storage

That means:

- If server restarts → files are deleted
- Disk storage is not permanent (unless using paid persistent disk)

So already this is risky.

💰 Does Storing File on Render Increase Cost?

There are 2 main cost factors:

1 Storage Cost

If Render provides persistent disk:

- You pay per GB of storage
 - More files = more disk usage
 - More disk usage = higher plan needed
-

2 Bandwidth Cost (Very Important)

Bandwidth = Data transferred from server to users

Example:

User uploads 5MB image.

Then 1000 users view it.

Total outgoing data:

Code

That 5GB is counted as **outbound bandwidth**.

Most cloud providers charge for:

- Outbound traffic
- Sometimes inbound is free

More users → more bandwidth → higher bill

◆ 2 If You Use AWS S3 or ImageKit

Instead of:

User → Render → Store on Render → User downloads from Render

You do:

User → Render → Upload to S3 →

User downloads directly from S3 CDN

Now what happens?

- ✓ Render handles only upload
- ✓ S3 handles storage
- ✓ CDN handles delivery

So:

- 👉 Your Render bandwidth reduces
- 👉 Render storage reduces

👉 Server load reduces

How Charges Actually Apply

Storage Charges

Charged per GB per month.

Example (approximate concept):

- 1GB stored for 1 month = small cost
 - 100GB stored = bigger cost
-

Bandwidth Charges

Charged per GB of data transferred OUT.

Example:

Scenario	Bandwidth Used
Upload 5MB file	Usually free or very cheap
1000 users download 5MB file	5GB outbound
10,000 users	50GB outbound

More traffic → more bandwidth → more cost

Why Bandwidth Becomes Large?

Because:

File size × Number of downloads = Bandwidth usage

Example:

- Profile picture = 2MB

- 10,000 users open profile page

Code

If hosted on Render:

Render pays that bandwidth

You get charged.

If hosted on ImageKit CDN:

CDN handles delivery

Much cheaper + optimized.

Real Production Best Practice

In real companies:

We NEVER store user images on backend server disk.

We:

1. Use memoryStorage in Multer
2. Upload file to:
 - S3
 - ImageKit
 - Cloudinary
3. Store only file URL in database

Example DB:

JavaScript

```
username "Meet"  
profilePic "https://ik.imagekit.io/xyz/profile123.jpg"
```

Now:

- ✓ Backend is lightweight
- ✓ Server restarts safe

- ✓ Scalable
 - ✓ Lower cost
-

Architecture Comparison

Bad (Costly & Risky)

User → Render → Save on Render Disk → Users download from Render

Problems:

- Ephemeral storage
 - Bandwidth cost on Render
 - Scaling issue
-

Good (Production Standard)

User → Render → Upload to S3/ImageKit

Users → Download from CDN

Benefits:

- Low server load
 - Cheap bandwidth
 - Auto scaling
 - Fast delivery
-

Final Clean Answer For You

If you store files on Render:

- ✓ Yes, storage + outbound bandwidth increases cost
- ✓ More downloads = more bandwidth
- ✓ Render plan may need upgrade

If you use S3 / ImageKit:

- ✓ Render cost reduces
 - ✓ Storage cost shifts to S3
 - ✓ Bandwidth handled by CDN
 - ✓ Much better architecture
-

Since you are building backend projects seriously, I strongly recommend:

- 👉 Use memoryStorage + upload to S3 or ImageKit
 - 👉 Store only URL in database
-

If you want, next I can explain:

- Exact flow of uploading to S3 using Multer
- What is CDN?
- What is pre-signed URL?
- How Instagram stores images?

Tell me 😊

now we have an access of file for now i use imagekit as an cloud storage i read docs of imagekit to load file server to cloud storage and also generate url and get as an response but i can't much understood from docs that tells me write

```
import ImageKit, { toFile } from '@imagekit/nodejs';
const client = new ImageKit({
  privateKey: process.env['IMAGEKIT_PRIVATE_KEY'], //
  This is the default and can be omitted
});
```

i think it mean it tells in which imagekit account should i load this file using private key
after that i have to do this

```
const file = await imagekit.files.upload({
  file: await toFile(Buffer.from(req.file.buffer), 'file'),
  fileName: 'Test',
});
```


i think this code will upload the file on account based on that private key using file buffer and we have this file buffer with help of multer like req.file.buffer and it will also take an file name i don't know what this toFile dose
and one more think i can't directly import package why ? i have to write something like this
`const ImageKit = require("@imagekit/nodejs");`
`const { toFile } = require("@imagekit/nodejs");`

Very good 🔥 you already understand 80% correctly.
Now I'll clear the remaining 20% step-by-step.

✓ 1 What does `privateKey` mean?

Yes ✓ you are correct.

JavaScript

```
const client = new ImageKit
  process. 'IMAGEKIT_PRIVATE_KEY'
```

This means:

👉 "Upload this file into the ImageKit account that owns this private key."

Every **ImageKit** account has:

- `publicKey`
- `privateKey`
- `urlEndpoint`

The **private key**:

- Authenticates your backend
- Allows secure uploads
- Should NEVER be exposed to frontend

So yes — this tells ImageKit **which account to upload into**.

**2**

What is happening in this code?

JavaScript

```
const file = await imagekit.  
  .await toFile Buffer. req. 'file'  
  'Test'
```

Let's break this clearly.

♦ Step 1: `req.file.buffer`

If using:

JavaScript

```
multer.
```

Then Multer gives:

JavaScript

```
req. .
```

This buffer contains the binary data of the uploaded file.

Example:

- Image
- PDF
- Video

It is just raw bytes stored in RAM.

♦ Step 2: What is `toFile()` ?

This is the part confusing you.

`toFile()` is a helper function provided by ImageKit SDK.

It converts:

👉 A Buffer

into

👉 A file format that ImageKit upload API understands

Think of it like:

Code

So:

JavaScript

```
Buffer.from(req.body.data)
```

creates a Buffer (raw file data)

Then:

JavaScript

```
buffer.toFile('file')
```

wraps it in a format compatible with ImageKit upload system.

You can think of `toFile()` as:

“Convert this memory data into an uploadable file object.”



3

What does `fileName` mean?

JavaScript

```
fileName: 'Test'
```

This is the name that will appear inside your ImageKit dashboard.

Better practice:

JavaScript

```
fileName: req.body.data
```

Or:

JavaScript

```
fileName Date. + "-" + req. .
```

✓ 4 Why can't you use `import` directly?

You wrote:

JavaScript

```
import ImageKit toFile from '@imagekit/nodejs'
```

But you had to write:

JavaScript

```
const ImageKit = require "@imagekit/nodejs"  
const          = require "@imagekit/nodejs"
```

This is because of:

🔥 ES Modules vs CommonJS

Node.js has two module systems:

1 CommonJS (Default in most projects)

Uses:

JavaScript

```
require  
module.
```

Your project is probably using this.

That's why:

JavaScript

```
const ImageKit = require "@imagekit/nodejs"
```

works.

2 ES Modules (Modern syntax)

Uses:

JavaScript

```
import
```

If you want to use `import`, you must:

Add in package.json:

JSON

```
"module"
```

OR rename file to:

Code

Otherwise Node throws error.

So nothing is wrong.

It's just module system difference.

Full Flow (Your Final Understanding)

User uploads file →

Multer stores in memory →

You get:

JavaScript

```
req. .
```

Then:

1. Convert buffer using `toFile()`
2. Upload using privateKey account
3. ImageKit returns response

4. You store:

JavaScript

file.

in database

Then send URL to frontend.



Example Clean Code (Production Style)

JavaScript

```
const ImageKit = require("@imagekit/nodejs")
const { privateKey } = require("@imagekit/nodejs")

const imagekit = new ImageKit({
  publicKey: process.env.IMAGEKIT_PUBLIC_KEY,
  privateKey: process.env.IMAGEKIT_PRIVATE_KEY,
  urlEndpoint: process.env.IMAGEKIT_URL_ENDPOINT
})

app.post("/upload", upload.single("photo"), async (req, res) => {
  try {
    const uploadedFile = await imagekit.upload({
      file: req.file.buffer,
      fileName: req.file.filename,
      date: Date.now() + "-" + req.file.filename
    })

    res.status(200).json({
      message: "Uploaded successfully",
      url: uploadedFile.url
    })
  } catch (error) {
    res.status(500).json({
      message: "Upload failed"
    })
  }
})
```



Final Clear Answers To Your Questions

- ✓ Yes, privateKey decides which ImageKit account
- ✓ Yes, file buffer comes from Multer

- ✓ `toFile()` converts buffer into uploadable file object
 - ✓ `require()` vs `import` is CommonJS vs ES module issue
-

You're actually understanding cloud architecture very well 🧠

If you want next level:

- What happens internally when uploading?
- Why is direct upload from frontend better?
- What is signed upload?
- How Instagram-like apps handle this?

Tell me 😊

ES module are older ? or commonjs is new ?

Good question 🔥 This confusion is very common.

👉 **CommonJS is older.**

👉 **ES Modules (ESM) are newer.**

Timeline

1 CommonJS (Older)

- Introduced around **2009**
- Created specifically for **Node.js**
- Uses:

JavaScript

```
const express = require "express"  
module.      = something
```

At that time, JavaScript (browser JS) did NOT have a module system.

So Node created its own → **CommonJS**

2 ES Modules (Newer / Modern)

- Officially introduced in ES6 (2015)
- Standard JavaScript module system
- Works in:
 - Browsers
 - Node.js
 - Modern bundlers

Uses:

JavaScript

```
import express from "express"
export default something
```

This is the **official JavaScript standard module system**.



So Why Does Node Still Use CommonJS?

Because:

- Node was built before ES Modules existed
- Millions of projects already use CommonJS
- Backward compatibility is important

Later, Node added ES Module support.



Which One Should You Use Today?

For new projects:

👉 Use ES Modules (import/export)

Because:

- It's standard JavaScript
- Works everywhere
- Future-proof

But:

👉 Most backend tutorials still use CommonJS

So you will see `require()` everywhere.

Quick Comparison

Feature	CommonJS	ES Modules
Older?	✅ Yes	❌ No
Standard JS?	❌ No	✅ Yes
Browser support?	❌ No	✅ Yes
Modern projects?	⚠️ Still used	✅ Recommended

Simple Rule

If your `package.json` has:

JSON

```
"module"
```

→ Use `import`

If not:

→ Use `require`

Final Answer

👉 CommonJS = older Node module system

👉 ES Modules = newer official JavaScript standard

Since you're building modern backend with cloud storage, I recommend:

✓ Start using ES Modules

✓ Add `"type": "module"`

✓ Practice with `import`

If you want, I can also explain:

- Why `__dirname` doesn't work in ES modules
- Difference between default export and named export
- How bundlers like Vite/Webpack handle modules

Just tell me 😊

What happens internally when uploading?

Why is direct upload from frontend better?

What is signed upload?

How Instagram-like apps handle this?

Excellent 🔥 Now you're asking **real production architecture** questions.

Let's go step-by-step like a real system design interview.

1 What Happens Internally When Uploading?

Let's say user uploads an image.

🧠 Flow (Normal Backend Upload)

User → Backend → Cloud Storage

Step-by-step:

1. User selects file in browser
2. Browser creates `multipart/form-data`
3. File is sent as binary stream
4. Your server receives it
5. Multer parses it
6. You get:

7. You send that buffer to **ImageKit** or **Amazon Web Services S3**
 8. Cloud storage saves file
 9. Cloud returns a URL
 10. You store URL in database
-

What's Actually Happening at Network Level?

Files are not sent at once.

They are sent in:

👉 Small chunks (stream)

Example:

- 10MB file
- Sent in small packets over TCP

Your server:

- Receives packets
- Reconstructs file
- Processes it

That's why large uploads consume:

- Memory
 - CPU
 - Bandwidth
-

Why Is Direct Upload From Frontend Better?

Now let's see smarter architecture.

Traditional Way

User → Backend → S3

Problem:

- Backend bandwidth increases
 - Backend CPU increases
 - Backend memory increases
 - Slower
-

✓ Direct Upload Architecture

User → S3 (directly)

But securely.

Flow:

1. Frontend asks backend:
"Give me permission to upload"
2. Backend generates signed upload config
3. Frontend uploads directly to S3/ImageKit
4. File stored
5. Backend only stores returned URL

Now:

- ✓ Backend does NOT handle file
 - ✓ No heavy bandwidth on server
 - ✓ Scalable
 - ✓ Cheaper
-

3 What Is Signed Upload?

Very important concept 🔒

You cannot expose:

JavaScript

privateKey

to frontend.

So how do we allow frontend to upload securely?

We create:

- 👉 A signed request
- 👉 Temporary permission

Backend generates something like:

```
JSON
{
  "signature": "12345678",
  "expires": "..."
}
```

This signature proves:

- Backend allowed this upload
- Valid only for limited time
- Cannot be reused

Then frontend sends:

File + signature → ImageKit/S3

Cloud verifies signature

If valid → upload allowed

🔥 Why Is This Secure?

Because:

- Private key stays in backend
- Frontend gets temporary permission only
- Even if someone steals signature, it expires

4 How Instagram-like Apps Handle This?

Big companies NEVER upload through main backend servers.

Example architecture (simplified):

User → Upload Service → Object Storage → CDN → Users

Let's break it.

Real Production Setup

Storage:

- Amazon Web Services S3
- Or Google Cloud Storage

CDN:

- CloudFront
- Akamai
- Fastly

Flow:

1. User requests upload token
 2. Backend generates signed upload URL
 3. User uploads directly to S3
 4. S3 stores file
 5. CDN caches file globally
 6. Other users download from nearest CDN edge server
-

Why This Is Powerful?

If 1 million users view image:

- 👍 Backend is not touched
- 👍 S3 + CDN handle everything

This saves:

- Backend bandwidth
 - Backend CPU
 - Backend memory
 - Money
-

Visual Comparison

Backend Upload

User



Backend (heavy load)



S3

Direct Signed Upload

User



S3 directly



Backend only stores URL

Much cleaner.



Final Architecture Recommendation For You

Since you're using **ImageKit**:

Best practice:

1. Backend generates authentication parameters
2. Frontend uploads directly
3. Backend stores returned URL

Do NOT:

- Upload large files through your Render server
 - Store files on Render disk
-



Final Summary

◆ Internally:

Files are streamed in chunks → processed → stored → URL returned.

◆ **Direct Upload:**

Reduces server load + reduces bandwidth + scalable.

◆ **Signed Upload:**

Temporary secure permission to upload without exposing private key.

◆ **Instagram-like apps:**

Use object storage + CDN + microservices.

You're now entering system design level thinking *100*

If you want next level:

- How to handle 1 million concurrent uploads?
- How WhatsApp stores media?
- What is multipart upload in S3?
- How video platforms store large files?

Tell me 😊